2026 SHENZHEN FINTECHATHON | INTERNATIONAL TRACK | TOPIC C: CROSS-BORDER & STUDENT FINANCE ASSISTANT

AuraRoute AI

Technical Documentation: System Architecture, Core Algorithms and Security Design

An explainable, read-only, non-custodial, compliance-first advisory agent that plans cross-border tuition payments. Theme: AI as a Financial Participant.

Submission Item 01 | Team AuraRoute AI | Final version, 20 October 2026

15,240 USD	Over quota	1,260 USD	10 Dec 2026
12,000 GBP invoice, converted by the read-only exchange-rate tool	Only 8,000 USD of the annual quota is left. Advice: use the bank's documented route; never split	Balance after paying, below the 2,500 USD illustrative framework baseline	Latest safe start for a 15 Dec due date, with a 5-day clearing buffer

Result of the recorded offline run on the synthetic sample invoice (Item 06). Test suite: 29 unit tests, all passing. Payment or write endpoints in the code: 0.

How this document maps to the scoring criteria

Criterion (weight)	What a reviewer can verify	Where
Task completion (40%)	The agent completes a real financial task end to end: it reads an invoice, has the student confirm every value, converts the currency, tracks the annual quota, computes the latest safe start date, checks the post-payment balance and returns advice that cites its sources. One command reproduces the run offline.	1.1, 1.4, 1.7 Items 04 and 06
Security & compliance (30%)	No money movement. Identifiers are redacted before parsing or any model call. Model output is untrusted and schema-checked. The human gate is enforced twice. Over-quota amounts are directed to the bank's official channel, never split. Hash-chained audit log. Permission Tiers and a 14-item Known-Risk List with residual ratings.	1.3, 1.6 Item 05
Innovation & interaction (30%)	A guardrail instead of an optimizer. A bounded AI role gated by deterministic code. Verified data and rule queries routed on separate paths. Provenance on every output (data mode, rule citations). A confirmation step and a dashboard that explain each result.	1.2, 1.3, 1.8

1.1 System Summary: AI as a Financial Participant

Paying university tuition abroad is a task every international student has to complete correctly, usually under a deadline, and three sets of rules collide while doing it. First, an individual in China may purchase foreign exchange up to an annual quota of USD 50,000 under SAFE rules; above it, a bank asks for supporting documents (Bank of China lists the passport with a valid visa, the school's admission letter and the school's tuition or living-cost certificate). Second, SAFE has acted against splitting a payment to stay under limits, so the obvious workaround is also the dangerous one. Third, a transfer takes days to clear and a late payment can trigger an enrolment hold, while the visa may still require proof of funds after the invoice is paid. Students today piece this together from bank pages, forums and guesswork.

AuraRoute AI takes this task on as a financial participant. It reads the invoice, asks the student to confirm what it read, retrieves the relevant rules with their sources, checks the numbers with read-only tools and returns a plan the student can act on. It does not touch the money. It is an explainable, compliance-first advisory agent: every statement in its

output can be traced to a computed value or to a cited rule.

The task, stated precisely

Input	Processing	Output
Tuition invoice text. Student profile (home country, annual quota already used). Account balances, read-only.	Redact identifiers, extract fields, student confirmation, rule retrieval, currency conversion, quota check, latest-start-date calculation, balance check, audit entry.	Invoice amount in USD. Quota verdict and remaining quota. Latest safe initiation date. Projected balance and baseline warning. Advisory notes with cited rules. Data-source label (`fx_source`).

Four design principles

- **Advisory, not transactional.** The system is non-custodial and read-only. It never initiates, alters, splits or routes a payment, and it makes no legal or immigration determination.
- **Human-verified inputs.** No parsed value reaches the compliance logic until the student has confirmed it. The check is made at the gate and again at the orchestrator.
- **Rules decide, language explains.** Compliance results come from deterministic code and cited rule text. The language model can read an unfamiliar invoice layout; it never decides anything.
- **Official channels only.** When an amount exceeds the remaining quota, the system flags it and directs the student to the bank's documented tuition-payment route. It never suggests splitting.

> **What AuraRoute AI deliberately does not do**
>
> It does not hold, move, split or route funds. It does not give legal or immigration determinations. It does not guess a missing field: a value the parser cannot find stays empty until the student supplies it. It does not claim integration with any bank backend: the only external services are Fixer.io (exchange rates) and Plaid Sandbox (balances), with recorded fixtures when no credentials are set.

1.2 Component Architecture and LLM Role

The system is a pipeline of four numbered layers with a human validation gate between the first and second. External tools attach to the orchestrator only. Figure 1 shows the architecture. The orange path carries the verified payload straight to Layer 3; only a rule query goes to the knowledge base.

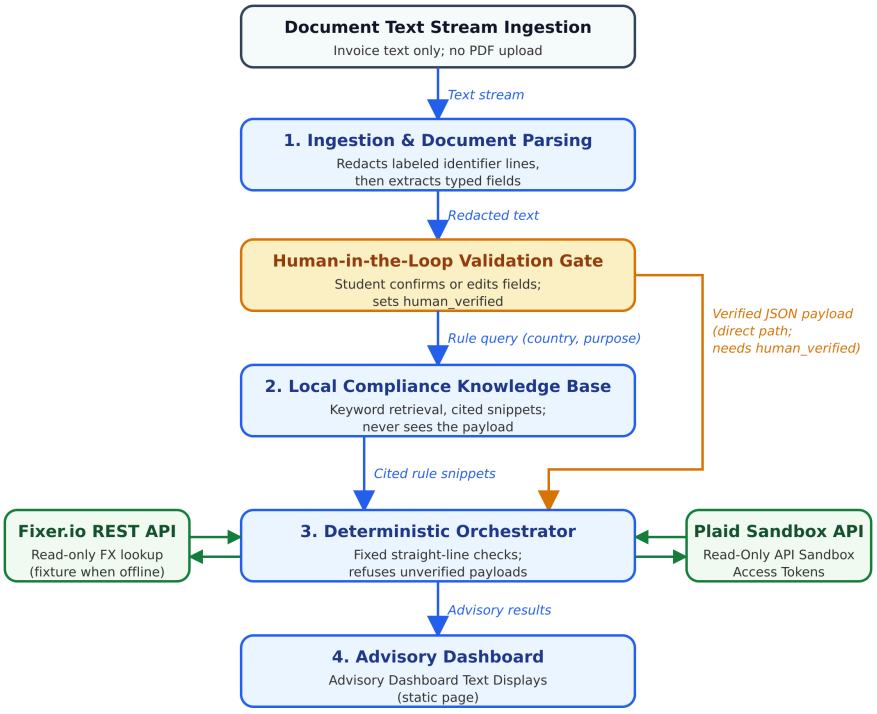


Figure 1. System architecture. The orange path carries the verified payload directly to Layer 3.

Figure 1. System architecture. Layer 4 is a static advisory page; the prototype has no charts and no real-time alerts.

Layer	Receives	Produces	Code	Trust position
1. Ingestion and Document Parsing	Invoice as a text stream (the prototype reads text, not PDF files)	Redacted text and a typed record: amount, currency, due date, institution, confidence, `human_verified = False`	`parse_and_sanitize_invoice`	Treats the document as untrusted input
Human validation gate	Typed record and the student's confirmation	Record with `human_verified = True`, or a refusal with a reason	`human_in_the_loop_gate`	Only the student can verify
2. Local Compliance Knowledge Base	Rule query: home country and topic keywords. No amount, date or identity	Up to five snippets, each with id, source type, source, section, URL and a "verify" note	`retrieve_rules`	Static packaged corpus; never sees the payload
3. Deterministic Orchestrator	Verified payload, profile, snippets, exchange rates, balances	Advisory result object and an audit entry	`execute_compliance_routing_check`	The only component with tool access; no model inside
4. Advisory Dashboard	Advisory result	Static HTML page with escaped values and a "planning information only" banner	`render_dashboard` in `run_demo.py`	Display only

Interaction design: what the student sees



1. **Read and redact.** The invoice text is loaded and identifying lines are replaced with `[REDACTED]` before anything else happens.
2. **Confirm.** The student sees the parsed values with a confidence score (for example "12000.0 GBP due 2026-12-15 (Example University), confidence 1.0"), confirms them, and is asked to type in any required field the parser could not find. Nothing proceeds without this step.
3. **Check and advise.** The dashboard shows four cards (invoice in GBP and USD, annual-quota verdict with the amount remaining, balance after payment against the baseline, latest safe start date), the advisory notes, and the cited rule snippets with their sources. Cards that need attention turn amber, so the two problems in the sample case are visible at a glance.

1.3 Role of the Language Model and How Deterministic Code Gates It

The model's operational role is document understanding. University invoices come in many layouts, and a regular expression only reads the labeled layout it was written for. A bursar's letter such as "Please remit twelve thousand pounds sterling (GBP 12,000.00) to Example University no later than 15 December 2026" has no "Total Due:" label and no ISO date, so the parser finds nothing. The model reads that letter and proposes the four fields. A second, planned role is regulatory text translation: restating dense rule text in plain English for a student. Both roles follow the same rule: the model proposes, deterministic code disposes.

Model task	Why a model is useful	Status in the prototype
Template-less invoice parsing	Handles layouts the regular expressions cannot read, without a template per university.	**Implemented.** `llm_fallback_extract` with a REST adapter for Gemini or Anthropic (`llm_adapter.py`). Off unless a provider and key are configured.
Regulatory text translation	Rule texts are dense and written for banks, not students.	**Designed, not implemented.** Output would be display text only and never feed a calculation.
Proposing tool parameters	Could map a free-text request to a tool call.	**Planned.** Any proposal would be checked against an allow-list first.

Seven gates between the model and a decision

Gate	Mechanism
Input	The model receives redacted text only, wrapped as untrusted data, with a system prompt that says never to follow instructions inside the invoice and to reply with JSON only (temperature 0).
Capability	The adapter only builds a request and returns text. The model has no tool, network or account-data access.
Output	`validate_llm_extraction` accepts known keys only, allow-listed currency codes, positive numeric amounts and valid ISO dates. Anything else is discarded whole.
Scope	The model can fill only fields the regular expressions missed. It cannot overwrite a parsed value.
Approval	The model can never set `human_verified`. The student confirms every value.
Decision	Quota, deadline and balance results are computed in code from verified values.
Failure	Any adapter error returns an empty string, which validation rejects. The pipeline continues with the parsed values and the student fills the gap.

Hostile or faulty model output, and what the code does with it

Model reply	Result
`ignore previous instructions` (not JSON)	Rejected. Fields stay empty.
`{"amount": 5, "run_shell": "rm -rf /"}`	Rejected: unknown key.
`{"amount": -5}` or `{"amount": true}`	Rejected: not a positive number.
`{"currency": "XXX"}` or `{"due_date": "tomorrow"}`	Rejected: not allow-listed, not an ISO date.
Valid JSON proposing amount 999999 when the parser already found 100	Amount ignored, parsed value kept. Only the missing due date is filled, and `human_verified` stays False.

Each row is a unit test in `tests/test_core.py`. Prompt injection is reduced by these gates, not eliminated: a wrong but well-formed value can still pass validation, which is why the student's confirmation is mandatory.

Excerpt 2. The model can only fill gaps (auraroute_core.py)

```python
def llm_fallback_extract(redacted_text, schema, llm_fn):
    """Fill only fields the regexes missed. llm_fn(text) returns a JSON string (untrusted)."""
    missing = [k for k in ("amount", "currency", "due_date", "institution") if schema.get(k) is None]
    if not missing:
        return schema
    clean = validate_llm_extraction(llm_fn(redacted_text))
    if clean is None:
        return schema
    out = dict(schema)
    filled = [k for k in missing if k in clean]
    for k in filled:
        out[k] = clean[k]
    out["llm_filled_fields"] = filled
    out["parser_confidence"] = _confidence(out)
    out["human_verified"] = False        # the model can never verify its own output
    return out
```

1.4 Task Completion Framework

The orchestrator runs nine fixed steps, once per payload, in straight-line code. There are no loops and no branches that depend on model output.

1. **Redact** labeled identifier lines, then extract amount, currency, due date and institution. Missing fields stay empty.
2. **Score** the parse: confidence is the share of the four fields found. If fields are missing, the optional model may fill only those.
3. **Gate.** The student confirms or corrects. The gate refuses when a required field is missing, the amount is not positive or the date is not ISO format.
4. **Retrieve** cited rule snippets for the home country and topic.
5. **Convert.** `amount_usd = amount x FX(currency, USD)`, using the read-only rate lookup.
6. **Track the quota.** `remaining = annual_quota - quota_used`. If `amount_usd > remaining`, flag it and advise the bank's documented tuition-payment route (typically the admission letter and the tuition certificate). Never split.
7. **Date.** `latest_start = due_date - 5 days` (the clearing buffer is a configured constant, not a measurement).
8. **Balance.** `projected = sum(balances in USD) - amount_usd`. Warn if it is below the 2,500 USD illustrative framework baseline. That figure is far below many real requirements, and living costs vary by country, city and year, so the note tells the student to confirm requirements with the institution or consulate.
9. **Audit and return.** Append a hash-chained entry and return the advisory result.

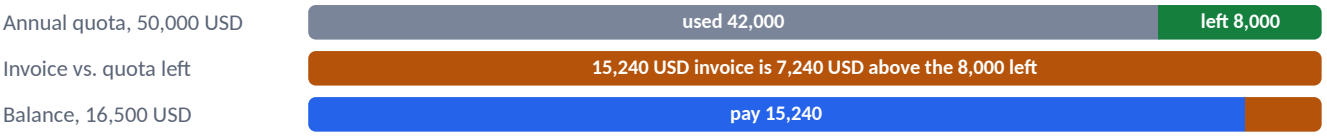
Excerpt 4. Quota tracking and balance baseline inside the orchestrator (auraroute_core.py)

```
    country = rules["country_rules"].get(profile.get("home_country"))
    if country:
        remaining = country["annual_quota_usd"] - profile.get("annual_quota_used", 0.0)
        result["remaining_quota_usd"] = round(remaining, 2)
        result["within_standard_quota"] = amount_usd <= remaining
        if amount_usd <= remaining:
            result["advisory_notes"].append("Amount is within the remaining annual quota.")
        else:
            result["advisory_notes"].append(
                "Amount exceeds the remaining standard annual quota. Ask your bank about the "
                "documented tuition-payment route, which typically requires the admission "
                "letter and the tuition certificate. Do not split the payment.")
    else:
        result["advisory_notes"].append("No quota rule loaded for this country; check manually.")

    total_usd = sum(b["amount"] * fx_fn(b["currency"], "USD") for b in balances_fn())
    result["total_balance_usd"] = round(total_usd, 2)
    result["projected_balance_usd"] = round(total_usd - amount_usd, 2)
    if total_usd - amount_usd < rules["minimum_visa_maintenance_usd"]:
        result["visa_balance_warning"] = True
        result["advisory_notes"].append(
            "Projected post-payment balance is below the illustrative framework baseline, which is far below many real
requirements. "
            "Review your maintenance-fund requirements with your institution or consulate.")
```

Worked scenario (synthetic data, recorded fixtures)

A student from China has used 42,000 of the 50,000 USD annual quota and holds 16,500 USD in two sandbox accounts (9,500 checking, 7,000 savings). The invoice is 12,000 GBP due 15 December 2026. At the recorded rate of 1.27 the invoice is 15,240 USD.



Third bar: 1,260 USD remain after paying (7.6% of the balance), below the 2,500 USD illustrative framework baseline.

Check	Computation	Result and advice
Currency	12,000 x 1.27	15,240.00 USD. `fx_source` = fixture.
Annual quota	50,000 - 42,000 = 8,000 remaining; 15,240 > 8,000	Flagged. "Ask your bank about the documented tuition-payment route, which typically requires the admission letter and the tuition certificate. Do not split the payment."
Balance	16,500 - 15,240 = 1,260; 1,260 < 2,500	Warning: below the illustrative framework baseline; review maintenance-fund requirements with the institution or consulate.
Deadline	15 Dec - 5 days	Latest safe initiation date: 10 December 2026.

Edge cases and the test that proves each

Situation	Behavior	Test
Invoice layout not recognized	Fields stay empty, confidence below 1.0; the model may fill gaps; the student must confirm.	`test_missing_fields_stay_none`, `test_messy_invoice_end_to_end_with_redacted_text_only`
Missing field, amount not positive, non-ISO date	The gate refuses with an explicit message.	`test_gate_refuses_*`

Payload not verified	The orchestrator raises and writes `BLOCKED_UNVERIFIED_PAYLOAD`.	`test_orchestrator_blocks_unverified_payload_and_logs_it`
Within the remaining quota	Note says so; no warning.	`test_within_quota`
Over the remaining quota	Flag, bank-route advice, "Do not split the payment."; no split action exists in the result.	`test_over_quota_advises_bank_route_and_never_splits`
No rule loaded for the country	"Check manually"; verdict is None, not a guess.	`test_unknown_country_asks_for_manual_check`
Live exchange-rate call fails	Recorded fixture used, `FX_LIVE_FAILED_USING_FIXTURE` logged, result labeled `fixture-fallback`.	`test_fx_failure_falls_back_to_fixture`
An earlier audit entry is altered	`verify_audit_log` returns False.	`test_audit_log_chain_detects_tampering`

Excerpt 3. The human validation gate (auraroute_core.py)

```python
def human_in_the_loop_gate(schema, user_confirmation=None):
    """Nothing passes downstream until the user has confirmed every required field."""
    if user_confirmation is None:
        return schema
    verified = dict(schema)
    for key in REQUIRED_FIELDS + ("institution",):
        if key in user_confirmation:
            verified[key] = user_confirmation[key]
    missing = [k for k in REQUIRED_FIELDS if verified.get(k) in (None, "")]
    if missing:
        raise ValueError(f"Gate refused: missing required fields {missing}")
    if float(verified["amount"]) <= 0:
        raise ValueError("Gate refused: amount must be positive")
    datetime.datetime.strptime(verified["due_date"], "%Y-%m-%d")  # raises on bad format
    verified["human_verified"] = True
    return verified
```

1.5 Data Pipeline Routing and Synchronization

The pipeline keeps the student's transaction data and the rule lookup on separate paths. After the gate, the verified payload goes directly to Layer 3 and is the single source of truth for amount, currency and due date. In parallel, Layer 2 receives a query made only of the home country and topic keywords (in the recorded run: *"CN tuition annual quota splitting visa maintenance funds"*). It contains no amount, date or identity, so the knowledge base can neither read nor alter the payload. Layer 2 returns cited snippets (in the recorded run R3, R5, R1, R4 and R2) and Layer 3 combines both into one result.

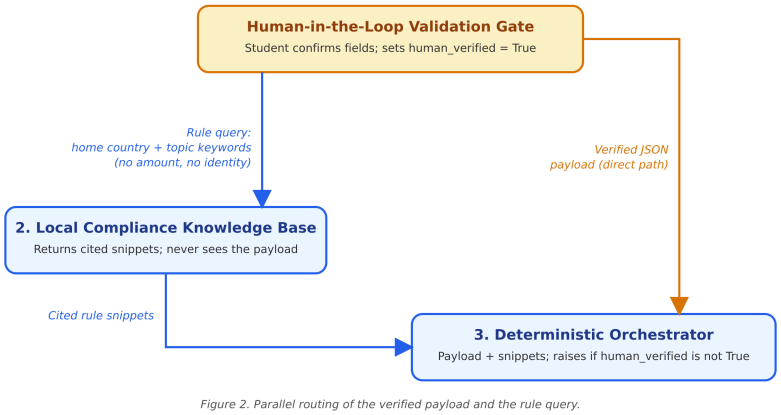


Figure 2. Parallel routing of the verified payload and the rule query.

Figure 2. Parallel routing of the verified payload and the rule query.

The orchestrator checks the `human_verified` flag itself rather than trusting the gate. A payload that skips the gate is refused, and the refusal is written to the audit log before the error is raised.

1.6 Security & Compliance Infrastructure

Compliance-first behavior

- **Anti-structuring by design.** Our rule base cites SAFE's 2009 action on fund splitting, and the code never produces a split plan. Over the quota the output is a flag plus a pointer to the bank's documented channel.
- **Cited, checkable rules.** Every snippet carries its source type (official SAFE, bank guidance or GOV.UK), section, URL and a note on what to verify. Several sources date from 2007 to 2009, and the notes say so (Appendix B).
- **Advisory framing.** The dashboard banner reads "Advisory planning information only. This tool does not move money or make legal determinations."

Controls and the evidence for each

Control	How it works	Function	Evidence
Read-only, non-custodial	No payment or write endpoint. The only external calls are an exchange-rate read and a balance read; the one POST is Plaid's balance read.	`fetch_fx_rate`, `fetch_sandbox_balances`	Code review; Item 05, Tier 0 and 1
Redaction before parsing	Eight labeled identifier types are removed first: name, passport number, phone, email, government or national ID, address, student ID, date of birth. The pattern consumes the whole line, so multi-word names go entirely, while "Institution:" lines survive.	`parse_and_sanitize_invoice`	Redaction tests in both test files; log lines in Item 06
Human approval gate	Values are inert until confirmed. Orchestrator re-checks and refuses.	`human_in_the_loop_gate`, `execute_compliance_routing_check`	Gate and blocked-payload tests
Untrusted-input handling	Invoices and model replies are treated as untrusted; strict schema and allow-list.	`validate_llm_extraction`	Hostile-output tests
Secrets	Keys come from environment variables. `.env` is git-ignored; only `.env.example` is committed.	`run_demo.py`, wrappers	Repository review

Output safety	Dashboard values pass through `html.escape`, so text from an invoice cannot inject markup.	`render_dashboard`	Code review
Tamper-evident audit	Hash-chained entries (below).	`audit_log`, `verify_audit_log`	Chain test; verified chain in the run log

Excerpt 1. Redaction pattern (auraroute_core.py)

```
# Labeled-line redaction: each matching line is consumed whole, so multi-word values are removed.
# Covers names, passport numbers, phone numbers, email addresses, government/national ID numbers,
# plus address, student ID and date of birth. Unlabeled identifiers are NOT caught (known limitation).
PII_PATTERN = re.compile(
    r"(?im)^[ \t]*(Passport No:|Name:|Phone:|Email:|Government ID:|National ID:|ID Number:"
    r"|Address:|Student ID:|Date of Birth:)[^\n]*")
```

Permission Tiers (summary; full table in Item 05)

Tier	Permitted	Not permitted
0. Money movement	Nothing.	Holding, moving, routing or splitting funds; writing to any payment endpoint.
1. External read-only	Exchange-rate lookup (Fixer.io); balance read (Plaid Sandbox, read-only API sandbox access tokens).	Any other endpoint or write; any call made by the model.
2. Local processing	Redaction, parsing, retrieval, checks, audit logging on redacted data.	Handling unredacted identifiers; persisting raw invoice text.
3. Human approval	The student confirms or corrects values.	Any downstream use of unconfirmed values.
AI model (optional)	Fill missing extraction fields from redacted text.	Tools, network, overwriting values, verifying a payload, changing a result.

Audit trail

Each entry stores a timestamp, an event name, the SHA-256 hash of the event details (never the details), the previous entry's hash and its own hash. Three events are recorded: `ROUTING_CHECK_COMPLETED`, `BLOCKED_UNVERIFIED_PAYLOAD` and `FX_LIVE_FAILED_USING_FIXTURE`. Changing or deleting an earlier entry breaks the chain and `verify_audit_log` returns False. The log is tamper-evident, not tamper-proof: someone with file access can delete it entirely.

Excerpt 5. Hash-chained audit entry (auraroute_core.py)

```
    digest = hashlib.sha256(json.dumps(details, sort_keys=True).encode()).hexdigest()
    entry = {"ts": datetime.datetime.now(datetime.timezone.utc).isoformat(),
             "event": event, "details_sha256": digest, "prev_sha256": prev}
    entry["entry_sha256"] = hashlib.sha256(json.dumps(entry, sort_keys=True).encode()).hexdigest()
```

Known-Risk List

Item 05 publishes 14 risks with mitigations and residual ratings. The ones we want reviewers to see first: prompt injection is reduced, not eliminated (Medium); unlabeled identifiers can pass redaction (Medium); the rule corpus is small and partly dated 2007 to 2009 (Medium); the 2,500 USD baseline is not an official figure (Medium); the prototype has no authentication, which would be High if deployed as a service.

1.7 Execution Evidence and Reproducibility

Everything above can be reproduced offline, with the Python standard library only, in three commands:

```
python -m unittest discover -s tests -v     # 29 tests: 25 core and adapter, 4 supplementary
python run_demo.py --auto-confirm            # writes execution_evidence_logs.txt, dashboard.html,
audit.log
python run_demo.py                           # interactive: you confirm the parsed values
```

Artifact	What it shows	Item
`execution_evidence_logs.txt`	Mode line (FIXTURE or LIVE), the redacted text with all eight identifier labels replaced, the parsed record, the refused unverified payload, the simulated gate, the retrieved rule ids, every computed figure, the two advisory notes and "Audit log hash chain verified: True".	06
`dashboard.html`	The four cards, advisory notes and cited rules for the sample case.	06
`audit.log`	Two hash-chained entries for the run: the blocked payload and the completed check.	06
`tests/test_core.py`, `tests/test_extended_redaction.py`	Behavior of every control described in this document.	04

With `FIXER_API_KEY` and `PLAID_ACCESS_TOKEN` (plus Plaid credentials) set, the same run uses Fixer.io and Plaid Sandbox and the log's Mode line reads LIVE. The adapters for Gemini and Anthropic are tested against mocked responses; the live model path depends on the team's own key and the provider's current API.

1.8 Innovation & Interaction

- **A guardrail, not an optimizer.** Most payment tools help a user get money across faster. AuraRoute AI is built so that the cheapest-looking workaround, splitting, is never offered.
- **A bounded AI role.** The model reads, code decides, the student approves. Seven gates are listed in section 1.3 and each is tested.
- **Separate paths for data and rules.** The rule query carries no amount or identity, and the payload never visits the knowledge base.
- **Provenance on every output.** The result says where its exchange rate came from (`live`, `fixture` or `fixture-fallback`), which rules it relied on and what to verify about each.
- **Interaction that explains.** The confirmation step shows confidence, and the dashboard marks exactly which check needs attention and why.

Roadmap

Next step	Why it matters
Verified rule corpus with article numbers	Replaces paraphrased entries and removes the 2007 to 2009 dating caveat.
Destination-specific maintenance figures	Replaces the illustrative 2,500 USD baseline.
Plaid Link, authentication, encryption at rest	Required before any use beyond a one-user local prototype.
Embedding-based retrieval, more countries	Better recall than keyword matching; extends beyond the CN quota rule.
Plain-English rule translation by the model	The second model role in section 1.3, under the same validation pattern.
Pilot with a university international office	Tests the confirmation step and wording with real students.

1.9 Known Limitations

- Redaction targets labeled lines. Identifiers in other formats may not be caught, so the confirmation screen should be reviewed before anyone shares a screenshot.
- The prototype reads invoice text, not PDF files or images, and its dashboard is a static page.
- The rule corpus is five paraphrased entries with keyword retrieval; Appendix B lists what to verify for each. The GOV.UK entry applies to UK-bound students.
- The USD 50,000 quota, the 2,500 USD baseline and the 5-day clearing buffer are configurable defaults, not verified current requirements.
- Sandbox and fixture data may not reflect a real student's accounts.
- The tool is advisory and does not replace guidance from the student's bank, institution or consulate.

Appendix A: Reference Implementation (`auraroute_core.py`)

The complete core module is reproduced below so a reviewer can read the logic without opening the repository. Section 1.4 explains the algorithm and the excerpts in sections 1.3 to 1.6 show the parts that matter most; the full listing sits here so the body of the document stays readable. The listing is identical to the file in Item 04.

```
"""AuraRoute AI: core advisory logic (reference implementation).

Advisory and read-only: nothing in this module moves money or writes to an external service.
"""
import datetime
import hashlib
import json
import os
import re

# ------------------------------------------------------------------ rules data
RULES_DATABASE = {
    "network_clearing_buffer_days": 5,          # configurable constant, not a prediction
    "minimum_visa_maintenance_usd": 2500.00,   # illustrative framework baseline, NOT an official requirement; real needs vary
by country and living costs
    "country_rules": {
        "CN": {
            "annual_quota_usd": 50000.00,      # SAFE individual annual FX quota
            "source": "SAFE individual FX guidance (verify current figure with your bank)",
        }
    },
}

REQUIRED_FIELDS = ("amount", "currency", "due_date")
ALLOWED_CURRENCIES = {"USD", "EUR", "GBP", "CNY", "AUD", "CAD", "JPY", "SGD", "HKD", "CHF"}
# Labeled-line redaction: each matching line is consumed whole, so multi-word values are removed.
# Covers names, passport numbers, phone numbers, email addresses, government/national ID numbers,
# plus address, student ID and date of birth. Unlabeled identifiers are NOT caught (known limitation).
PII_PATTERN = re.compile(
    r"(?im)^[ \t]*(Passport No:|Name:|Phone:|Email:|Government ID:|National ID:|ID Number:"
    r"|Address:|Student ID:|Date of Birth:)[^\n]*")
AUDIT_PATH = "audit.log"


# ------------------------------------------------- Layer 1: ingestion + redaction
def _confidence(schema):
    found = sum(schema.get(k) is not None for k in ("amount", "currency", "due_date", "institution"))
    return round(found / 4, 2)          # heuristic: share of fields matched


def parse_and_sanitize_invoice(raw_text):
    """Redact identifiers first, then extract typed fields. Missing fields stay None."""
    text = PII_PATTERN.sub(r"\1 [REDACTED]", raw_text)

    m_amount = re.search(r"Total Due:\s*[$€£]?\s*([\d,]+(?:\.\d{2})?)", text)
    m_currency = re.search(r"Currency:\s*([A-Z]{3})", text)
    m_date = re.search(r"Due Date:\s*(\d{4}-\d{2}-\d{2})", text)
    m_inst = re.search(r"Institution:\s*([^\n]+)", text)

    schema = {
        "amount": float(m_amount.group(1).replace(",", "")) if m_amount else None,
        "currency": m_currency.group(1) if m_currency else None,
        "due_date": m_date.group(1) if m_date else None,
        "institution": m_inst.group(1).strip() if m_inst else None,
        "human_verified": False,
    }
    schema["parser_confidence"] = _confidence(schema)
    return schema, text


# ------------------------------------------- LLM output validation (untrusted input)
def validate_llm_extraction(raw_output):
    """Accept model output only if it passes a strict schema and allow-list check."""
    try:
        data = json.loads(raw_output)
    except (TypeError, ValueError):
        return None
    allowed = {"amount", "currency", "due_date", "institution"}
    if not isinstance(data, dict) or set(data) - allowed:
        return None
    clean = {}
    if "amount" in data:
        a = data["amount"]
        if isinstance(a, bool) or not isinstance(a, (int, float)) or a <= 0:
            return None
        clean["amount"] = float(a)
    if "currency" in data:
        if data["currency"] not in ALLOWED_CURRENCIES:
            return None
        clean["currency"] = data["currency"]
```

```
    if "due_date" in data:
        try:
            datetime.datetime.strptime(str(data["due_date"]), "%Y-%m-%d")
        except ValueError:
            return None
        clean["due_date"] = str(data["due_date"])
    if "institution" in data:
        inst = data["institution"]
        if not isinstance(inst, str) or len(inst) > 120:
            return None
        clean["institution"] = inst.strip()
    return clean


def llm_fallback_extract(redacted_text, schema, llm_fn):
    """Fill only fields the regexes missed. llm_fn(text) returns a JSON string (untrusted)."""
    missing = [k for k in ("amount", "currency", "due_date", "institution") if schema.get(k) is None]
    if not missing:
        return schema
    clean = validate_llm_extraction(llm_fn(redacted_text))
    if clean is None:
        return schema
    out = dict(schema)
    filled = [k for k in missing if k in clean]
    for k in filled:
        out[k] = clean[k]
    out["llm_filled_fields"] = filled
    out["parser_confidence"] = _confidence(out)
    out["human_verified"] = False        # the model can never verify its own output
    return out


# ---------------------------------------------------- Human-in-the-loop gate
def human_in_the_loop_gate(schema, user_confirmation=None):
    """Nothing passes downstream until the user has confirmed every required field."""
    if user_confirmation is None:
        return schema
    verified = dict(schema)
    for key in REQUIRED_FIELDS + ("institution",):
        if key in user_confirmation:
            verified[key] = user_confirmation[key]
    missing = [k for k in REQUIRED_FIELDS if verified.get(k) in (None, "")]
    if missing:
        raise ValueError(f"Gate refused: missing required fields {missing}")
    if float(verified["amount"]) <= 0:
        raise ValueError("Gate refused: amount must be positive")
    datetime.datetime.strptime(verified["due_date"], "%Y-%m-%d")  # raises on bad format
    verified["human_verified"] = True
    return verified


# ------------------------------------------------ Layer 2: local rule retrieval
def retrieve_rules(query, corpus, top_k=3):
    """Keyword retrieval over a static local corpus. Each hit keeps its source and section."""
    terms = {w for w in re.findall(r"[a-z]+", query.lower()) if len(w) > 3}
    scored = []
    for item in corpus:
        hay = (item["text"] + " " + " ".join(item.get("tags", []))).lower()
        score = sum(1 for t in terms if t in hay)
        if score:
            scored.append((score, item))
    scored.sort(key=lambda pair: -pair[0])
    return [dict(item) for _, item in scored[:top_k]]


# ---------------------------------------------------- Read-only tool wrappers
FX_FIXTURE = {("GBP", "USD"): 1.27, ("EUR", "USD"): 1.08, ("USD", "USD"): 1.0}
FX_STATE = {"source": "fixture"}      # "live", "fixture" or "fixture-fallback"


def fetch_fx_rate(base, quote, api_key=None):
    """Read-only FX lookup. Live Fixer.io call if a key is set, recorded fixture otherwise."""
    if base == quote:
        return 1.0
    if api_key:
        try:
            import requests
            r = requests.get("https://data.fixer.io/api/latest",
                             params={"access_key": api_key, "symbols": f"{base},{quote}"},
                             timeout=10)
            r.raise_for_status()
            rates = r.json()["rates"]
            FX_STATE["source"] = "live"
            return rates[quote] / rates[base]
        except Exception as exc:       # network, auth or format failure
            audit_log("FX_LIVE_FAILED_USING_FIXTURE", {"error": type(exc).__name__})
            FX_STATE["source"] = "fixture-fallback"
            return FX_FIXTURE[(base, quote)]
    FX_STATE["source"] = "fixture"
    return FX_FIXTURE[(base, quote)]
```

```python
def fetch_sandbox_balances(access_token=None, fixture=None):
    """Read-only balance lookup (Plaid Sandbox). Returns [{'amount':..., 'currency':...}]."""
    if access_token:
        import requests
        r = requests.post("https://sandbox.plaid.com/accounts/balance/get", timeout=10, json={
            "client_id": os.environ["PLAID_CLIENT_ID"],
            "secret": os.environ["PLAID_SECRET"],
            "access_token": access_token})
        r.raise_for_status()
        return [{"amount": a["balances"]["available"] or 0.0,
                 "currency": a["balances"]["iso_currency_code"] or "USD"}
                for a in r.json()["accounts"]]
    return fixture or [{"amount": 4200.00, "currency": "USD"}]   # recorded sandbox fixture


# ---------------------------------------------------------------------- Audit log
def audit_log(event, details, path=None):
    """Append-only, hash-chained. Stores a hash of the details, never the details."""
    path = path or AUDIT_PATH
    prev = "0" * 64
    if os.path.exists(path):
        with open(path, encoding="utf-8") as f:
            lines = f.read().splitlines()
        if lines:
            prev = json.loads(lines[-1])["entry_sha256"]
    digest = hashlib.sha256(json.dumps(details, sort_keys=True).encode()).hexdigest()
    entry = {"ts": datetime.datetime.now(datetime.timezone.utc).isoformat(),
             "event": event, "details_sha256": digest, "prev_sha256": prev}
    entry["entry_sha256"] = hashlib.sha256(json.dumps(entry, sort_keys=True).encode()).hexdigest()
    with open(path, "a", encoding="utf-8") as f:
        f.write(json.dumps(entry) + "\n")


def verify_audit_log(path=None):
    """True if every entry links to the previous one and no entry was altered."""
    path = path or AUDIT_PATH
    prev = "0" * 64
    with open(path, encoding="utf-8") as f:
        for line in f:
            entry = json.loads(line)
            claimed = entry.pop("entry_sha256")
            recomputed = hashlib.sha256(json.dumps(entry, sort_keys=True).encode()).hexdigest()
            if entry["prev_sha256"] != prev or recomputed != claimed:
                return False
            prev = claimed
    return True


# ----------------------------------------- Layer 3: deterministic advisory check
def execute_compliance_routing_check(payload, profile, rules=RULES_DATABASE,
                                     fx_fn=fetch_fx_rate, balances_fn=fetch_sandbox_balances,
                                     rule_snippets=None):
    if not payload.get("human_verified"):
        audit_log("BLOCKED_UNVERIFIED_PAYLOAD", {"reason": "human_verified is not True"})
        raise ValueError("Unverified payload blocked at orchestrator interface.")

    amount_usd = payload["amount"] * fx_fn(payload["currency"], "USD")
    deadline = datetime.datetime.strptime(payload["due_date"], "%Y-%m-%d")
    buffer_days = rules.get("network_clearing_buffer_days", 5)

    latest_start = deadline - datetime.timedelta(days=buffer_days)
    result = {
        "amount_usd": round(amount_usd, 2),
        "fx_source": FX_STATE["source"],
        "within_standard_quota": None,
        "remaining_quota_usd": None,
        "visa_balance_warning": False,
        "total_balance_usd": None,
        "projected_balance_usd": None,
        "latest_initiation_date": latest_start.strftime("%Y-%m-%d"),
        "advisory_notes": [],
        "cited_rules": rule_snippets or [],
    }

    country = rules["country_rules"].get(profile.get("home_country"))
    if country:
        remaining = country["annual_quota_usd"] - profile.get("annual_quota_used", 0.0)
        result["remaining_quota_usd"] = round(remaining, 2)
        result["within_standard_quota"] = amount_usd <= remaining
        if amount_usd <= remaining:
            result["advisory_notes"].append("Amount is within the remaining annual quota.")
        else:
            result["advisory_notes"].append(
                "Amount exceeds the remaining standard annual quota. Ask your bank about the "
                "documented tuition-payment route, which typically requires the admission "
                "letter and the tuition certificate. Do not split the payment.")
    else:
        result["advisory_notes"].append("No quota rule loaded for this country; check manually.")

    total_usd = sum(b["amount"] * fx_fn(b["currency"], "USD") for b in balances_fn())
```

```
    result["total_balance_usd"] = round(total_usd, 2)
    result["projected_balance_usd"] = round(total_usd - amount_usd, 2)
    if total_usd - amount_usd < rules["minimum_visa_maintenance_usd"]:
        result["visa_balance_warning"] = True
        result["advisory_notes"].append(
            "Projected post-payment balance is below the illustrative framework baseline, which is far below many real
requirements. "
            "Review your maintenance-fund requirements with your institution or consulate.")

    audit_log("ROUTING_CHECK_COMPLETED", {"amount_usd": result["amount_usd"],
                                          "within_quota": result["within_standard_quota"]})
    return result
```

Appendix B: Rule Corpus (`data/regulatory_snippets.json`)

Each entry is a paraphrase, not a quotation, and carries the note a reviewer would need to check it against the official text.

Id	Source type	Source	Section	What to verify
R1	official SAFE	SAFE, Q&A on the Detailed Rules on Measures for the Administration of Individual Foreign Exchange (5 Jan 2007) https://www.safe.gov.cn/en/2007/0105/819.html	annual quota	2007 page; confirm the current quota and any later changes on safe.gov.cn.
R2	official SAFE	SAFE news item on the Circular Concerning Further Improving Administration of Foreign Exchange Settlement and Sale Business for Individuals (SAFE English site, 2009) https://www.safe.gov.cn/en/2009/1223/913.html	fund splitting	2009 news item about the circular, not the circular itself; read the circular and cite its article numbers.
R3	bank guidance (not SAFE)	Bank of China, Individual Foreign Exchange Purchasing (English site) https://www.boc.cn/en/pbservice/pb4/200806/t20080626_1323972.html	purchasing beyond the total annual amount: study abroad	Bank page dated 2008 and says SAFE's latest announcements prevail. Confirm requirements with your own bank.
R4	bank guidance (not SAFE)	Bank of China, Individual Foreign Exchange Purchasing (English site) https://www.boc.cn/en/pbservice/pb4/200806/t20080626_1323972.html	notes on the annual total amount	Same bank page; confirm with your bank.
R5	official UK government (GOV.UK)	GOV.UK, Student visa: money you need https://www.gov.uk/student-visa/money	proof of funds	Living-cost amounts are set in the Immigration Rules and change over time; read the GOV.UK page and its financial-evidence guidance (gov.uk/guidance/financial-evidence-for-student-and-child-student-route-applicants) for the current figure. Applies to UK-bound students; use the destination country's official page for other countries.